Week 12 Final Group Project Brief

Purpose & Overview

The final group project is the capstone assignment for CST4714. Teams of **3–4 students** will design and implement a production-ready database solution that addresses a realistic case study of their own choosing or one provided by the instructor. Each project must:

- demonstrate coherent data modeling choices (relational, document, or hybrid)
- deliver a working implementation using PostgreSQL, MongoDB, or a blended architecture
- surface core administration practices, including performance, security, backup, replication, and monitoring considerations
- culminate in both a written deliverable and an in-class presentation during Week 15

Team Formation & Workflow Expectations

- Confirm final team rosters by the end of Week 12 and share preferred communication channels.
- Schedule regular working sessions and stand-ups to divide ownership across analysis, design, implementation, and admin tasks.
- Maintain shared documentation (e.g., version control repo, shared drive) that captures decisions, meeting notes, risks, and lessons learned.
- Proactively surface blockers during lab/recitation or via instructor office hours for timely feedback.

Required Deliverables

1. **Design Document** – Executive summary, problem statement, personas/use cases, logical and physical data models, technology justification, and administrative architecture plan.
2. **Implementation Package** – Database creation scripts, sample data loads, schema migrations, stored procedures or application interface code, plus configuration artifacts for users/roles, security policies, and monitoring hooks.
3. **Administrative Demonstration** – Evidence (screenshots, logs, scripts, or recorded demo) of: scheduled backups and restore validation; security hardening (role-based access, encryption, auditing); replication or high-availability setup with failover validation.

5. **Written Report Submission** – Consolidated PDF/Word package of all documentation, code links, and runbooks submitted before the Week 15 class meeting.

Milestone Timeline (Weeks 12–15)

Week	Focus	Expected Work Products
12	Case study selection & scope	Team charter, problem statement, high-level architecture sketch
13	Detailed design & tooling	Data dictionary, ER/aggregation diagrams, admin plan draft
14	Build, test, and admin validation	Implementation scripts, backup & security walkthroughs
15	Final polish & presentation	Slide deck, rehearsal feedback, final report & submission

Sample Scenario Ideas

- **Atlas Mini CRM Auth Upgrade (Slide 10)** – Deploy a single MongoDB Atlas cluster with one `customers` collection, then create read-only and read/write custom roles, enable built-in auditing, and document how to rotate SCRAM credentials. Show screenshots/logs of successful logins and blocked access.
- **Supabase Micro Support Queue (Slides 2–4)** – Model a three-table ticket system (projects, tickets, comments) in Supabase, add indexes that support two core queries, and enforce row-level security so that each team only sees its own tickets. Provide SQL policies plus a short demo script.
- **Atlas Query Tune Lab (Slides 4–5, 11)** – Import a CSV of 2–3k "orders" into Atlas, capture slow query stats, add one compound index following ESR, and re-run `explain()` to compare execution stats. Summarize before/after latency and memory footprint in the report.
- **Weekend Backup Drill (Slide 9)** – Use either Supabase (pg_dump) or MongoDB Atlas (snapshot/export) to back up a tiny dataset, drop a table/collection on purpose, and restore it. Include the exact commands, timestamps, and validation query outputs.
- **Replica Set Sandbox (Slide 7)** – Spin up the Atlas free-tier replica set, trigger a controlled failover in the UI, and capture how the application connection string handles the change. Document replication lag metrics and the steps to recover normal operations.

- **Monitoring Posture Check (Slides 11, 13)** – Connect Atlas metrics or Supabase Observability to a lightweight dashboard that tracks CPU, connections, and slow queries. Set a threshold alert (email/Slack/webhook) and prove it fires by running a stress script for a few minutes.

Technical & Administrative Requirements (when applicable)

- Choose technology stack(s) aligned to the case study. PostgreSQL, MongoDB, or a hybrid approach must be justified with data access patterns, scaling goals, and administrative needs.
- Implement at least one automation for backup/restore and one for security or compliance (e.g., automated user provisioning, policy enforcement, or auditing job).
- Configure replication or an equivalent high-availability strategy (e.g., PostgreSQL streaming replication, MongoDB replica set) along with documented failover test results.
- Document monitoring and alerting hooks (system metrics, query performance dashboards, log aggregation) used to validate system health.
- Ensure all scripts/code are runnable on the lab environment, with environment variables and secrets handled securely.

Evaluation Criteria

Projects will be evaluated on: - **Technical depth & correctness** – schema quality, query performance, and operational robustness. - **Administrative excellence** – completeness of backup, security, replication, and monitoring implementations plus clarity of runbooks. - **Documentation quality** – readability, organization, reproducibility, and traceability to requirements. - **Presentation & storytelling** – clarity, pacing, visuals, demo effectiveness, and ability to answer technical questions. - **Team collaboration** – evidence of equitable contribution, communication structure, and professionalism.

Submission Checklist

- ☐ All deliverables packaged in a single folder (design doc, scripts, admin evidence, slides, report).
- ☐ README or runbook describing how to recreate the environment, run scripts, and validate admin tasks.
- ☐ Links to source control or repositories with commit history for each team member.
- ☐ Backup presentation copy shared with the instructor.

Questions? Reach out via email or office hours before Week 14 to leave enough time for adjustments.